



# Universidad Autónoma de Baja California

## Programación Orientada a Objetos

Colecciones en Java

Profesor: MC. Itzel Barriba Cázares

# Arreglos

- ▶ Un **arreglo (array)** puede contener múltiples valores del mismo tipo de datos simultáneamente.
- ▶ Un arreglo **es un objeto que puede almacenar un grupo de valores**, todos del mismo tipo.

# Declaración de un arreglo

- ▶ La **declaración** de un arreglo es:

```
tipoDato[] nombreArreglo;
```

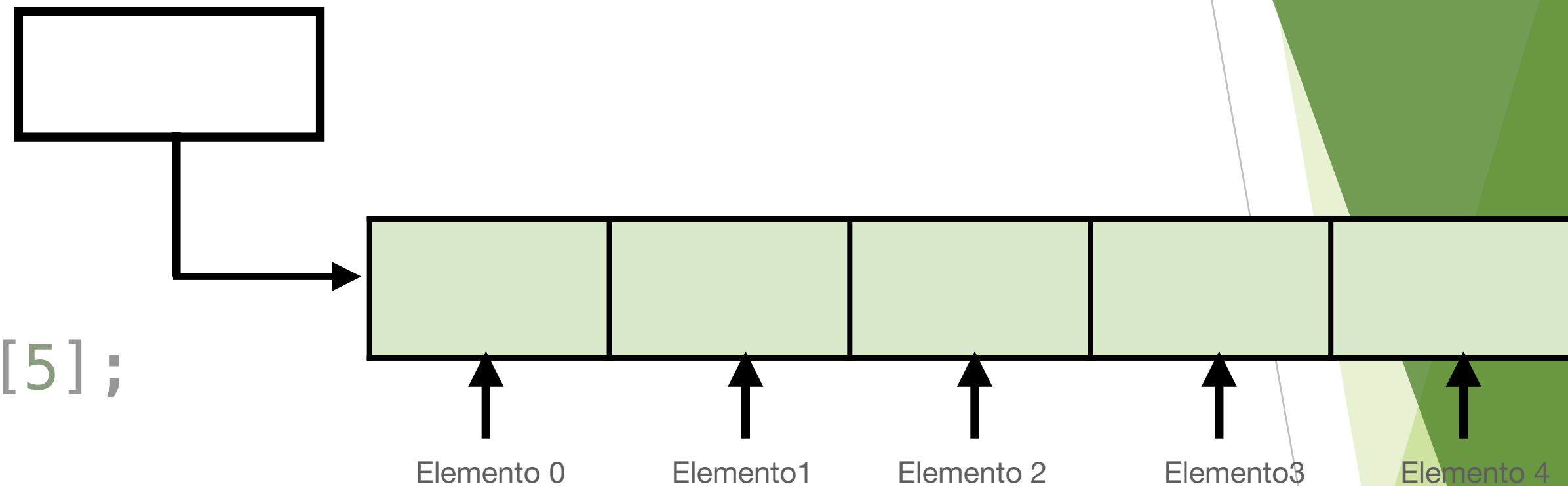
- ▶ Esta declaración declara nombreArreglo como una variable del tipoDato.
- ▶ Ejemplo:

```
int[] numeros;
```

- ▶ La variable números puede hacer referencia a un arreglo de valores int (enteros). **Los corchetes indican que esta es una referencia a un arreglo de enteros.**



# Declarar y crear una instancia de un arreglo



- ▶ La **asignación** de un arreglo es:

```
nombreArreglo = new tipoDato[5];
```

- ▶ Cuando se inicializa, también se debe establecer el numero de elementos
- ▶ Ejemplo:

```
numeros = new int[5];
```

- ▶ Es necesario utilizar la palabra new para crear un arreglo y asignar su dirección de memoria a la **variable numeros**.
- ▶ tipoDato: **puede ser del tipo (int, float, double, String, etc)**, el número dentro de los corchetes es el tamaño del arreglo. Indica cuantos elementos o valores puede contener el arreglo

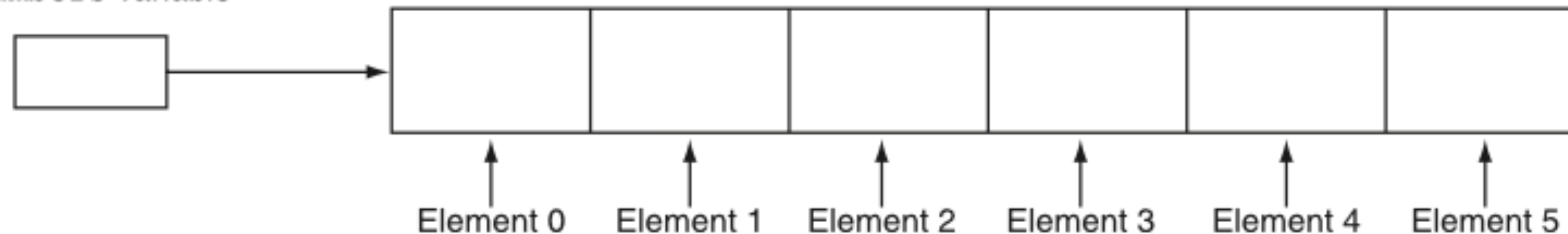
# Declarar y crear una instancia de un arreglo

- El arreglo se puede inicializar usando el operador new, de la siguiente manera:

```
int[] numeros = new int[5];
```

numbers references an array with enough memory for 6 int values

numbers variable



- El **arreglo numeros** tiene espacio para almacenar cinco elementos, donde el primer número se almacena en la posición del índice 0 y el último número se almacena en la posición 4.
- Los valores almacenados inicialmente serán los valores por default para el tipo de datos.

# Declarar y crear una instancia de un arreglo

- El arreglo se puede inicializar usando el operador new, de la siguiente manera:

```
int[] numeros = new int[5];
```

- Las siguientes declaraciones de arreglos son validas:

```
float[] temperatures = new float[100];  
char[] letters = new char[41];  
long[] units = new long[50];  
double[] sizes = new double[1200];
```

- El tamaño de un arreglo debe ser una expresión entera no negativa.



# Inicialización de un arreglo

- La inicialización más popular de un arreglo puede ser de una manera mas concisa:

```
float[] arregloPeso = {85f, 72f, 68f, 94f, 78f};
```

- La serie de valores dentro de las llaves y separados por coma se denomina lista de inicialización.



# Accesando a elementos de un arreglo

- Esto es posible porque a cada elemento se le asigna un número conocido como **subíndice**.

|    |    |    |    |    |                       |
|----|----|----|----|----|-----------------------|
| 85 | 72 | 68 | 94 | 78 | ← Valor del elemento  |
| 0  | 1  | 2  | 3  | 4  | ← Indices del arreglo |

- Un subíndice se utiliza como índice para señalar un elemento específico dentro del arreglo:

```
arregloPesos[subíndice];
```

- Un subíndice se utiliza como índice para señalar un elemento específico dentro del arreglo:

```
numbers[0] = 85;  
numbers[3] = 30;
```



# Ejemplo1 de un arreglo

```
public class BMICalculator {  
  
    // Método main.  
    public static void main(String[] args){  
        // inicializa tres arreglos de 5 elementos cada uno  
        float[] arregloPesos = new float[5];  
        float[] arregloAlturas = new float[5];  
        float[] arregloBMIs = new float[5];  
  
        // asigna los valores del arreglo de pesos  
        arregloPesos[0] = 85f;  
        arregloPesos[1] = 72f;  
        arregloPesos[2] = 68f;  
        arregloPesos[3] = 94f;  
        arregloPesos[4] = 78f;  
  
        //asigna los valores del arreglo de alturas  
        arregloAlturas[0] = 1.74f;  
        arregloAlturas[1] = 1.80f;  
        arregloAlturas[2] = 1.90f;  
        arregloAlturas[3] = 1.84f;  
        arregloAlturas[4] = 1.88f;  
  
        //calcula los BMIs y los guarda en el arreglo  
        arregloBMIs[0] = arregloPesos[0]/(arregloAlturas[0]*arregloAlturas[0]);  
        arregloBMIs[1] = arregloPesos[1]/(arregloAlturas[1]*arregloAlturas[1]);  
        arregloBMIs[2] = arregloPesos[2]/(arregloAlturas[2]*arregloAlturas[2]);  
        arregloBMIs[3] = arregloPesos[3]/(arregloAlturas[3]*arregloAlturas[3]);  
        arregloBMIs[4] = arregloPesos[4]/(arregloAlturas[4]*arregloAlturas[4]);  
  
        // Imprime el BMIs en la pantalla  
        System.out.println("El BMI de la persona 1 es: " + arregloBMIs[0] + ".");  
        System.out.println("El BMI de la persona 2 es: " + arregloBMIs[1] + ".");  
        System.out.println("El BMI de la persona 3 es: " + arregloBMIs[2] + ".");  
        System.out.println("El BMI de la persona 4 es: " + arregloBMIs[3] + ".");  
        System.out.println("El BMI de la persona 5 es: " + arregloBMIs[4] + ".");  
    }  
}
```

La salida del programa en Java es:

```
El BMI de la persona 1 es: 28.075043.  
El BMI de la persona 2 es: 22.222223.  
El BMI de la persona 3 es: 18.836565.  
El BMI de la persona 4 es: 27.76465.  
El BMI de la persona 5 es: 22.06881.
```

# Ejemplo2 de un arreglo

```
// Demonstrate a one-dimensional array.
class Array {
    public static void main(String[] args) {
        int[] mes_dias;
        mes_dias = new int[12];
        mes_dias[0] = 31;
        mes_dias[1] = 28;
        mes_dias[2] = 31;
        mes_dias[3] = 30;
        mes_dias[4] = 31;
        mes_dias[5] = 30;
        mes_dias[6] = 31;
        mes_dias[7] = 31;
        mes_dias[8] = 30;
        mes_dias[9] = 31;
        mes_dias[10] = 30;
        mes_dias[11] = 31;
        System.out.println("April tiene " + mes_dias[3] + " dias.");
    }
}
```

# Arreglos

- ▶ Java verifica estrictamente para asegurarse de que no intente almacenar o hacer referencia accidentalmente a **valores fuera del rango del arreglo**.
- ▶ Por ejemplo:

```
int[] values = new int[10];
```

- ▶ Java verificará el valor de cada índice en values esté entre 0 y 9. La comprobación de límites se realiza en tiempo de ejecución, el programa genera una excepción y finaliza de inmediato..



# Ejercicio

- Generar un arreglo de 10 elementos con valores del 0 al 9

```
public class GenerarArreglo {  
    public static void main(String[] args) {  
        int[] arreglo = new int[10];  
        int i;  
        for(i = 0; i < 10; i = i+1)  
            arreglo[i] = i;  
        for(i = 0; i < 10; i = i+1)  
            System.out.println("Arreglo[" + i + "]: " + arreglo[i]);  
    }  
}
```

|            |            |            |            |            |            |            |            |            |            |
|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|
| 0          | 1          | 2          | 3          | 4          | 5          | 6          | 7          | 8          | 9          |
| Arreglo[0] | Arreglo[1] | Arreglo[2] | Arreglo[3] | Arreglo[4] | Arreglo[5] | Arreglo[6] | Arreglo[7] | Arreglo[8] | Arreglo[9] |

La salida del programa en Java es:

```
Arreglo[0]: 0  
Arreglo[1]: 1  
Arreglo[2]: 2  
Arreglo[3]: 3  
Arreglo[4]: 4  
Arreglo[5]: 5  
Arreglo[6]: 6  
Arreglo[7]: 7  
Arreglo[8]: 8  
Arreglo[9]: 9
```

# Tamaño de un arreglo

- Los arreglos se implementan como objetos. El **tamaño de un arreglo**, es decir, la cantidad de elementos que puede contener un arreglo, se encuentra en su variable de instancia de longitud (**length**). Todos los arreglos tienen esta variable y siempre tendrá el tamaño del arreglo.

// Este programa muestra la longitud del arreglo.

```
class Length {  
    public static void main(String[] args) {  
        int[] a1 = new int[10];  
        int[] a2 = { 3, 5, 7, 1, 8, 99, 44, -10 };  
        int[] a3 = { 4, 3, 2, 1 };  
        System.out.println("length of a1 is " + a1.length);  
        System.out.println("length of a2 is " + a2.length);  
        System.out.println("length of a3 is " + a3.length);  
    }  
}
```

La salida del programa en Java es:

```
length of a1 is 10  
length of a2 is 8  
length of a3 is 4
```



# Ejercicio: Sumando los valores de un arreglo

- Calcular el promedio de los elementos de un arreglo

```
// Calcular el promedio de los valores en un arreglo.
public class Promedio {
    public static void main(String[] args) {
        double[] nums = { 10.1, 11.2, 12.3, 13.4, 14.5 };
        double resultado = 0;
        int i;
        for (i = 0; i < 5; i++)
            resultado = resultado + nums[i];
        System.out.println("El promedio es " + resultado / 5);
    }
}
```

# Longitud del arreglo

- Cada arreglo en Java tiene un campo publico llamado **length**. Este campo contiene la cantidad de elementos del arreglo.

```
double[] temperatures = new double[25];  
size = temperatures.length;
```



# Ciclo for mejorado

- La sintaxis es:

```
for (dataType elementVariable : array)  
    statement;
```

- Esta diseñado para iterar una vez por cada elemento de un arreglo. Cada vez que el ciclo itera, copia un elemento del arreglo a una variable.

```
int[] numbers = { 3, 6, 9 };  
for (int val : numbers)  
    System.out.println(val);
```

- Como el arreglo tiene 3 elementos el ciclo se repetirá tres veces.

# Ciclo for mejorado

- ▶ No puedes utilizar el ciclo for mejorado:
  - ▶ Si necesita cambiar el contenido del arreglo
  - ▶ Si necesitas trabajar con los elementos del arreglo en orden inverso
  - ▶ Si necesitas acceder a algunos de los elementos del arreglo, pero no a todos
  - ▶ Si necesitas trabajar simultáneamente con dos o más arreglos dentro del ciclo
  - ▶ Si necesitas hacer referencia al número de subíndice de un elemento en particular
- ▶ En cualquiera de estas circunstancias debes usar for tradicional

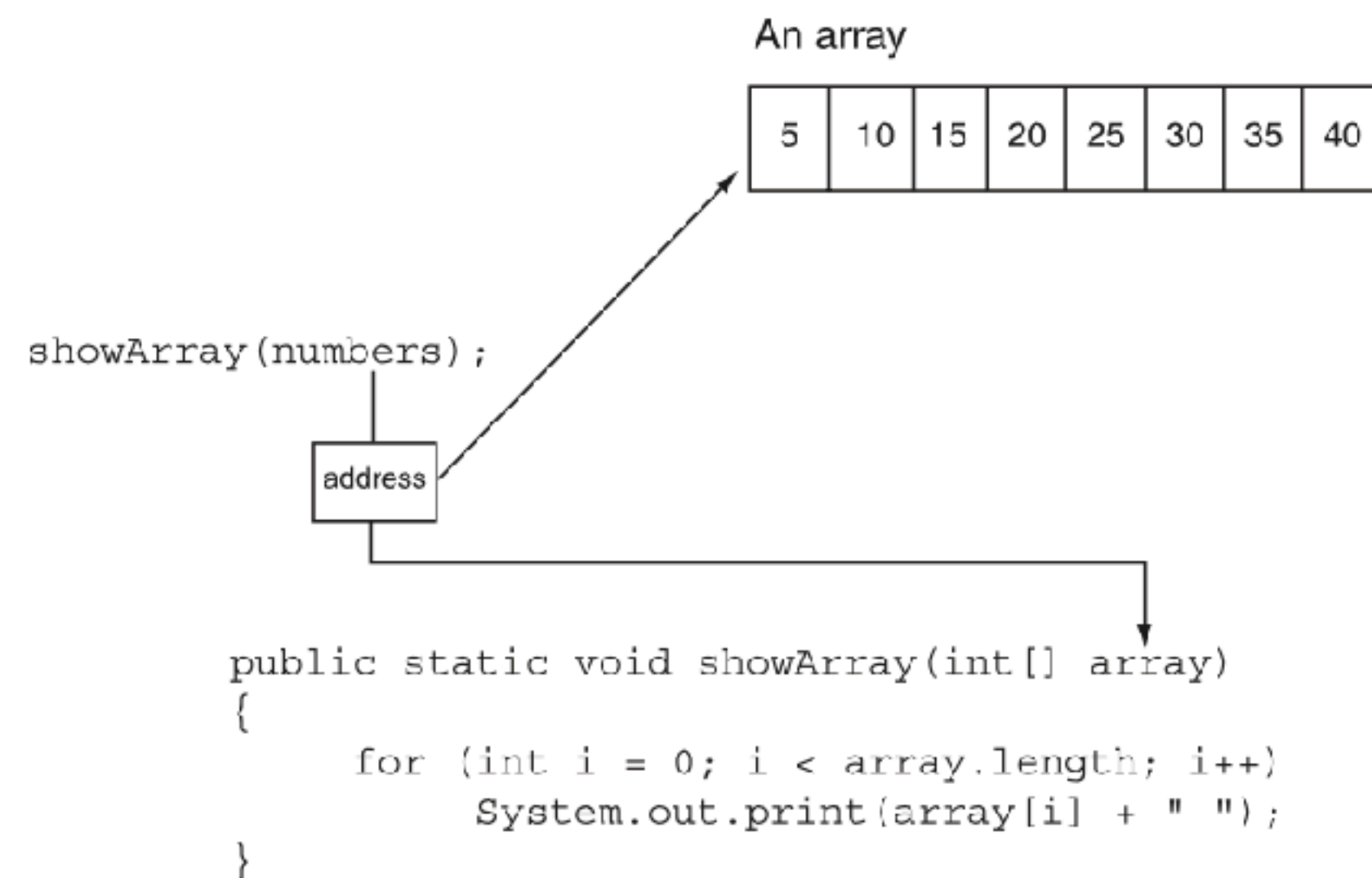
# Pase de arreglos como argumentos

- ▶ Un arreglo **puede ser pasado como un argumento hacia un método**. Para pasar un arreglo, debes pasar el valor en la variable que referencia el arreglo.
- ▶ Con frecuencia se necesitarán métodos que procesen los datos de los arreglos. Se pueden escribir métodos para almacenar valores en un arreglo, mostrar su contenido, sumar todos sus elementos, calcular su promedio, etc.
- ▶ Normalmente estos métodos aceptan un arreglo como argumento.



# Pase de arreglos como argumentos

- Cuando pasas un arreglo como argumento, simplemente se pasa el valor en la variable que hacer referencia al arreglo.



- Cuando se pasa un array completo a un método, se pasa igual que un objeto: no se pasa el array en sí, sino una referencia al array como parámetro.

# Pase de arreglos como argumentos

```
import java.util.Scanner;

/* This program demonstrates passing an array as an argument to a method.*/

public class PassArray {
    public static void main(String[] args) {
        final int ARRAY_SIZE = 4; // Size of the array
        // Create an array.
        int[] numbers = new int[ARRAY_SIZE];

        // Pass the array to the getValues method.
        getValues(numbers);

        System.out.println("Here are the " + "numbers that you entered:");

        // Pass the array to the showArray method.
        showArray(numbers);
    }

    private static void getValues(int[] array) {
        // Create a Scanner objects for keyboard input.
        Scanner keyboard = new Scanner(System.in);

        System.out.println("Enter a series of " + array.length + " numbers.");

        // Read values into the array
        for (int index = 0; index < array.length; index++) {
            System.out.print("Enter number " + (index + 1) + ": ");
            array[index] = keyboard.nextInt();
        }
    }

    public static void showArray(int[] array) {
        // Display the array elements.
        for (int index = 0; index < array.length; index++)
            System.out.print(array[index] + " ");
    }
}
```

# Regresar un arreglo desde un método

- Los métodos también pueden devolver un arreglo (es decir regresa una referencia a un arreglo).

Method header  $\longrightarrow$  `public static double[] getArray()`



### The method's return type

- El método `getArray()` devuelve un arreglo de valores dobles.



# Regresar un arreglo desde un método

```
/* This program demonstrates how a reference to an array can be returned from a method.
*/
```

```
public class ReturnArray {
    public static void main(String[] args) {
        double[] values;
        values = getArray();
        for (double num : values)
            System.out.print(num + " ");
    }

    public static double[] getArray() {
        double[] array = { 1.2, 2.3, 4.5, 6.7, 8.9 };
        return array;
    }
}
```

# Arreglos bidimensionales

- ▶ En Java, los **arreglos bidimensional** llamada matrices es un arreglo de arreglos. Que contiene renglones y columnas.
- ▶ Una matriz es utilizada para almacenar y trabajar con un conjunto de datos.

|       | Column 0 | Column 1 | Column 2 | Column 3 |
|-------|----------|----------|----------|----------|
| Row 0 |          |          |          |          |
| Row 1 |          |          |          |          |
| Row 2 |          |          |          |          |

# Declaración de una matriz

- ▶ La **declaración** de una matriz es:

```
tipoDato[][] nombreMatriz;
```

- ▶ Esto define una variable nombreMatriz, que es una matriz de dos dimensiones de tipoDato. Tenga en cuenta los corchetes ([][]), indican las dimensiones de la matriz.
- ▶ **tipoDato:** declara el tipo de dato de cada elemento contenido en el arreglo (int, float, double, String, etc) de los elementos del arreglo.

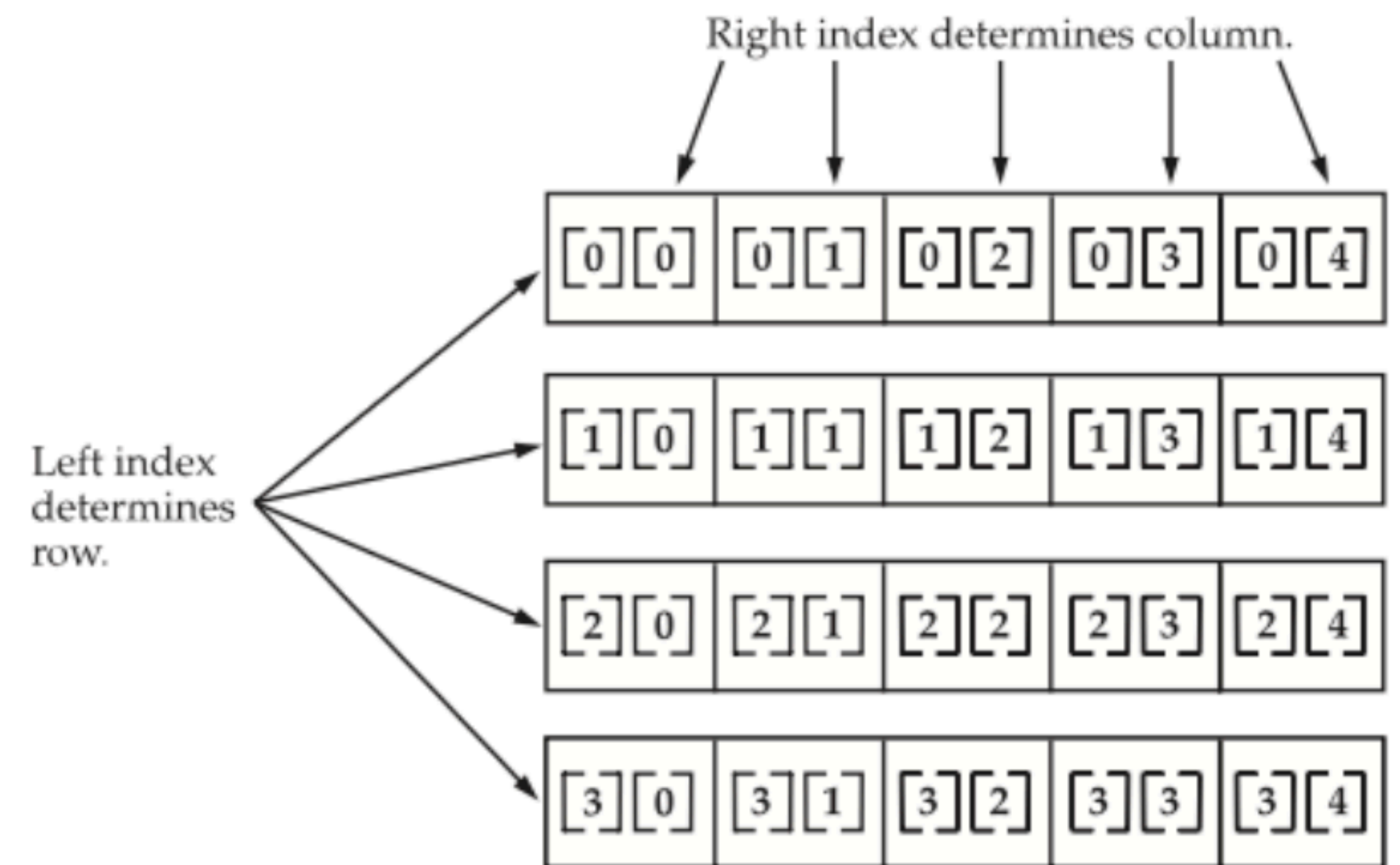


# Arreglos multidimensionales

- Por ejemplo, lo siguiente declara un **arreglo bidimensional** o **matriz de dos dimensiones**:

```
int[][] matriz = new int[4][5];
```

- Esto asigna una matriz de 4 por 5 elementos y la asigna a matriz. Internamente, esta matriz se implementa como un arreglo de arreglos de enteros (int):



# Asignación de una matriz

- Declaración de una matriz de 4 renglones por 5 columnas elementos de tipo entero:

```
int[] matriz = new int[4][5];
```

- Una vez definido la matriz se deben asignar los valores que contendrá de la siguiente manera:

```
matriz[0][0] = 1;  
matriz[0][1] = 2;  
matriz[0][2] = 3;  
matriz[0][3] = 4;  
matriz[0][4] = 5;  
matriz[1][0] = 6;  
matriz[1][1] = 7;  
matriz[1][2] = 8;  
matriz[1][3] = 9;  
matriz[1][4] = 10;  
matriz[2][0] = 11;  
matriz[2][1] = 12;  
matriz[2][2] = 13;  
matriz[2][3] = 14;  
matriz[1][4] = 15;  
matriz[3][0] = 16;  
matriz[3][1] = 17;  
matriz[3][2] = 18;  
matriz[3][3] = 19;  
matriz[1][4] = 120;
```

- La matriz con los valores de pesos es visualizado en la siguiente figura.

# Asignación de un arreglo

- La inicialización más popular de un arreglo puede ser de una manera mas concisa:

```
int[][] matriz = {{1, 2, 3, 4, 5},{6, 7, 8, 9, 10},{11, 12, 13, 14, 15},{16, 17, 18, 19, 20}};
```

| 0  | 1  | 2  | 3  | 4  | ← Columnas           |
|----|----|----|----|----|----------------------|
| 1  | 2  | 3  | 4  | 5  | ← Valor del elemento |
| 6  | 7  | 8  | 9  | 10 |                      |
| 11 | 12 | 13 | 14 | 15 |                      |
| 16 | 17 | 18 | 19 | 20 |                      |

↑ Renglones



# Acceso a un elemento de una matriz

- Una vez que se haya asignado una matriz, se puede acceder a un elemento específico de la matriz **especificando sus índices entre corchetes**.

The diagram illustrates a 3x4 matrix. To the left of the matrix, there are labels for rows: 'row 0', 'row 1', and 'row 2'. Above the matrix, there are labels for columns: 'column 0', 'column 1', 'column 2', and 'column 3'. A box labeled 'address' has an arrow pointing to the first cell of the matrix (row 0, column 0). Each cell in the matrix contains a two-dimensional array index notation, such as 'scores[0][0]' for the top-left cell.

|       | column 0     | column 1     | column 2     | column 3     |
|-------|--------------|--------------|--------------|--------------|
| row 0 | scores[0][0] | scores[0][1] | scores[0][2] | scores[0][3] |
| row 1 | scores[1][0] | scores[1][1] | scores[1][2] | scores[1][3] |
| row 2 | scores[2][0] | scores[2][1] | scores[2][2] | scores[2][3] |

- En Java todas las matrices se asignan dinámicamente. Todos los índices de las matrices empiezan en cero. Por ejemplo, para acceder al elemento 1 del arreglo, se hace de la siguiente forma:

```
matriz[0][1];
```

# Ejemplo1 de una matriz

```
public class EjemploMatriz {  
    // declara e inicializa una matriz  
    public static void main(String[] args){  
        int[][] matriz={{1, 2, 4},{2, 6, 8},{10, 20, 30}};  
        // imprime algunos elementos de la matriz  
        System.out.println("Elemento en el renglon 0 y columna 1 es: " + matriz[0][1] + ".");  
        System.out.println("Elemento en el renglon 2 y columna 2 es: " + matriz[2][2] + ".");  
        System.out.println("Elemento en el renglon 2 y columna 1 es: " + matriz[2][1] + ".");  
        System.out.println("Elemento en el renglon 1 y columna 0 es: " + matriz[1][0] + ".");  
    }  
}
```

La salida del programa en Java es:

```
Elemento en el renglon 0 y columna 1 es: 2.  
Elemento en el renglon 2 y columna 2 es: 30.  
Elemento en el renglon 2 y columna 1 es: 20.  
Elemento en el renglon 1 y columna 0 es: 2.
```

# Desplegar todos los elementos de una matriz

- El siguiente programa numera cada elemento de la matriz de izquierda a derecha, de arriba a abajo, y luego muestra sus valores.

```
public class MatrizDosDimensiones {  
    public static void main(String[] args) {  
        int[][] matriz = new int[4][5];  
        int i, j, k = 0;  
        for (i = 0; i < 4; i++)  
            for (j = 0; j < 5; j++) {  
                matriz[i][j] = k;  
                k++;  
            }  
        for (i = 0; i < 4; i++) {  
            for (j = 0; j < 5; j++)  
                System.out.print(matriz[i][j] + " ");  
            System.out.println();  
        }  
    }  
}
```

La salida del programa en Java es:

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |



# Longitud de una matriz

- Un arreglo bidimensional tiene varios campos de longitud. Uno que contiene el número de renglones y cada renglón tiene un campo de longitud que contiene el número de columnas.

```
public class Lengths
{
    public static void main(String[] args)
    {
        // Declare a 2D array with 3 rows and 4 columns.
        int[][] numbers = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 } };
        // Display the number of rows.
        System.out.println("The number of " + "rows is " + numbers.length);

        // Display the number of columns in each row.
        for (int index = 0; index < numbers.length; index++)
        {
            System.out.println("The number of " + "columns in row " + index + " is "
+numbers[index].length);
        }
    }
}
```

# Pase de matriz como argumentos

- ▶ Cuando una matriz se pasa a un método, el parámetro debe declararse como referencia a una matriz bidimensional.

```
private static void showArray(int[][] array)
```

- ▶ El parámetro de este método, array se declara como referencia a una matriz de enteros. Pasando matrices bidimensionales a métodos Cualquier matriz int bidimensional puede pasarse como argumento al método.